

DMP'26 Project Proposal

The Apprentice Project: Native GovTech Learning App for Public Education Systems

Oval17: Anurag Singh



1. Basic Details

Name:

Anurag Singh

Contacts & Information:

Email: anuragsingh7051@gmail.com

GitHub Profile: [Oval17](#)

Twitter: [Oval](#)

Location: Jaipur, Rajasthan, India

Time zone: Indian Standard Time (UTC+5:30)

Education Details:

Currently, I am in my last year of academic pursuit, enrolled in a double major program encompassing Computer Science and Physics at **BITS Pilani**.

2. Share Links, of your previous work on opensource projects.

For the past 17-18 months, I have actively engaged in contributing to open-source projects. Along this journey, I have acquired valuable knowledge spanning embedded systems, system design, object-oriented programming, machine learning, Arduino, and Raspberry Pi.

Additionally, I have tackled numerous algorithmic challenges on platforms such as Codeforces and LeetCode, nurturing a keen aptitude for programming and problem-solving in a broader context.

I have worked with SugarLabs in my 2nd Year under GSoC '24 and completed one of their flagship project which was to bring Sugar OS on RPi : [Details](#)

Projects/Repository/Pull Requests Links	Description
Konva-Drawboard	A simple and interactive drawing board built with React and Konva.js.
Point-Cloud Data Viewer	A modern 3D point cloud visualization tool built with React, TypeScript, and Three.js
Advanced Staggared View	A React TypeScript application that demonstrates advanced image loading with service worker caching and staggered loading techniques.
Compilation of my C projects	I keep working on different C projects in my Hobby time to get deeper understanding of systems
Tiled	Tiled is a general purpose tile map editor for all tile-based games, have been contributing to it.

I have made over 800+ **contributions** on GitHub, including more than **100+ pull requests** and involvement in over **60+ issue resolutions**, both active and resolved. My contributions have extended across various organizations and projects.

I have interned with TI AI and worked on making of various internal tools there.

3. Convince us that you will be a good fit for this project.

3.1 Codebase Familiarity

I have already explored TAP's GitHub organization (theapprenticeproject) to understand the existing stack:

- **Ai repo** (Frappe-based conversational engine) —> I've read the routing architecture and webhook patterns
- **RagService** (Python RAG microservice) —> the data source the AI tutor will consume
- **TAP-Lms-TAP-Trans** —> the combined LMS + translation layer the native app will surface to students

- **evalix web** —> the JS-based evaluation frontend whose data models will inform mobile content schema

3.2 Relevant Technical Depth

Requirement	My background
React Native	Built 3 production React projects (TypeScript); React Native shares the same component/state paradigm. I have set up Expo + bare RN projects locally and understand the bridge layer.
Frappe REST API	Contributed to a Django-based project (Learning Unlimited / ESP-Website) with 20+ merged PRs; deeply familiar with token-based API auth, resource endpoints, and serialization.
Web Sockets	Implemented real-time bidirectional state in Konva-Drawboard (WebSocket + React state reconciliation)
Offline Sync	Built the Advanced Staggered View project around Service Worker caching; understand cache invalidation, background sync, and conflict resolution
Low-literacy UX	GSoC '24 work required designing for users with no prior OS knowledge, enforcing the same "voice-first, icon-first" UX principle this project needs

3.3 Community

I actively contribute to PR reviews, issue triage, and community discussions (I have been actively involved with various FOSS organisation). I show up for async collaboration and respond quickly to mentor feedback.

Here are the pull requests I have generated for TAP till now (I am working. Actively to contribute more)

Pull Request Link	Description	Status
<u>#6</u>	fix(file-naming): sync Frappe File record after physical file move	Open
<u>#7</u>	docs: add CONTRIBUTING.md with setup, code style, and PR guide	Open

#11	Docs/add claude md	Open
---------------------	--------------------	------

Here are the pull requests I have generated which are merged to showcase my contribution to different FOSS organisation

Pull Request Link	Description	Status
#4201	Added pytest-django + pytest-xdist for parallel test execution improving Performance by almost 40 percent	Merged
#4237	Fix/email status button overflow on text droplets	Merged
#4234	Fix npm installation: re-add npm to packages_base.txt	Merged
#4205	Hide common searches for user types not available in the program	Merged
#4016	Bypass secure_required HTTPS redirect when DEBUG is True	Merged
#4015	Pre-select 'Student' in user selector when viewing student schedules.	Merged
#4004	Autofill nametag title with group name when 'Other group' is selected.	Merged
#4002	Fix theme editor formatting: move closing divs outside circles-only.	Merged
#4001	Fix extra b's in LaTeX output by decoding bytes to str	Merged
#4000	Add tabindex to class flag create/edit form elements	Merged
#3999	Rename Phase Zero to Program Lottery in user-facing text.	Merged
#4358	Scripting: Add optional defaultValue and toolTip params to Dialog add widget methods	Merged
#4338	Fix spacebar panning not working after focus leaves the map view	Merged
#4209	Upgrade Knockout.js 2.3.0 → 3.5.1 and Select2 3.4.3 → 4.1.0	Merged

#4344	macOS: Add Cmd+Shift+[and Cmd+Shift+] shortcuts to switch tabs	Merged
-----------------------	---	--------

Made over **50+ Quality Pull Requests and** thoroughly resolved Issues in LU , Tiled , CrowdFlux , TIAI and more. I have understood different parts of codebase, starting from very basics and understanding the whole flow architecturally.

4. Project Details

Problem Statement

India has ~260 million students in government schools. Existing EdTech products are built for private-school demographics: high-literacy parents, stable WiFi, flagship smartphones. TAP Buddy currently runs as a web app on Frappe LMS, a solid foundation, but one that fails government-school students because:

- 1. **No offline support** : classrooms lose connectivity mid-session; progress is lost
- 2. **Browser UX friction** : low-digital-literacy students cannot navigate URL bars, browser history, or desktop tabs
- 3. **No AI tutor real-time streaming** : web polling adds latency; a native WebSocket connection delivers sub-300 ms response times
- 4. **No whitelabeling** : NGO/state-government partners must fork the entire codebase to co-brand
- 5. **No DIKSHA interoperability** : the national content platform carries millions of pieces of curriculum-aligned content; ignoring it means reimplementing what already exists

This proposal describes how I will build a native Android app that solves all five problems over 13 weeks.

Part 1: Architecture

1.1 Tech Stack Decision: Expo (Managed Workflow) vs Bare React Native

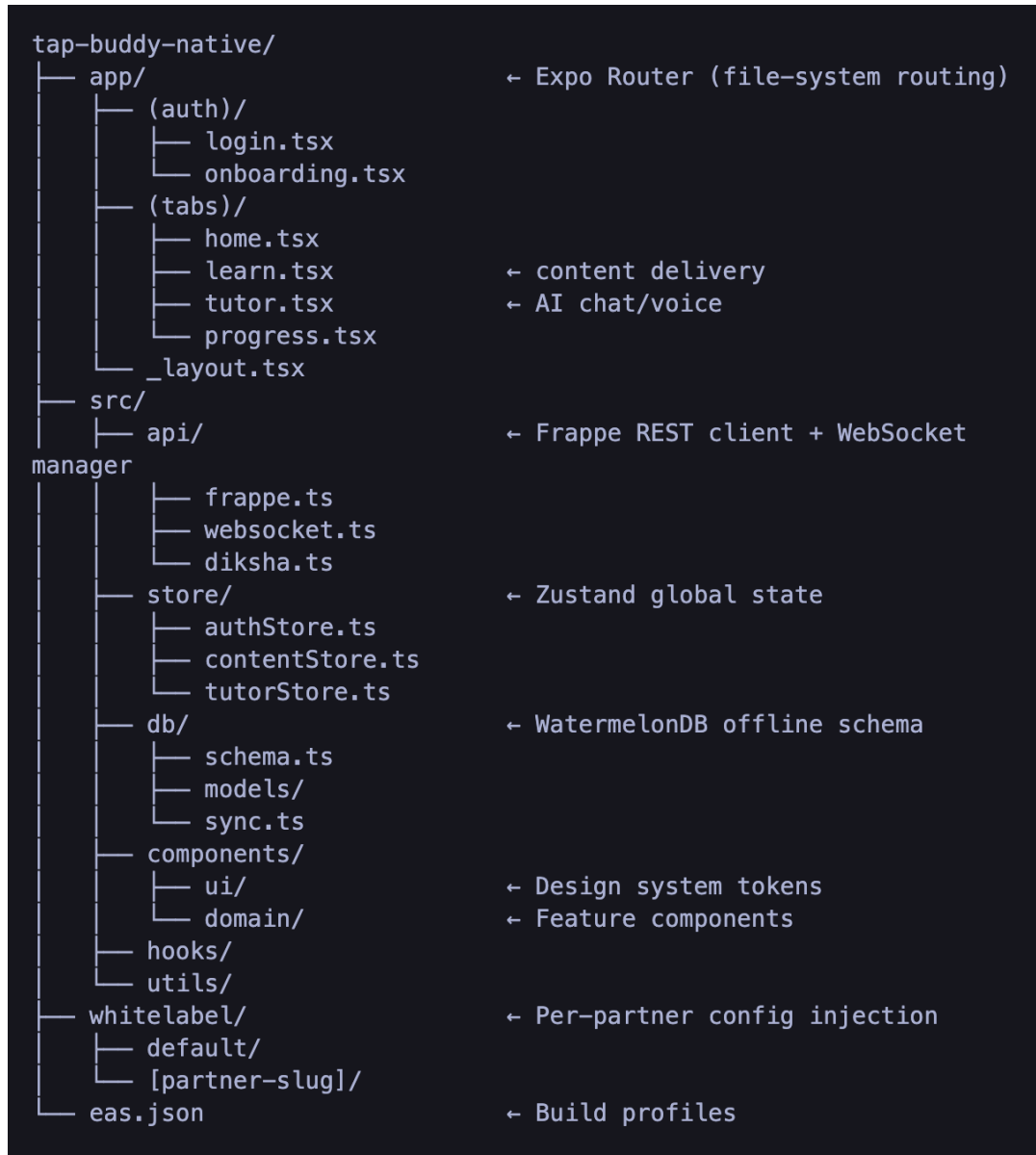
Factor	Expo Managed	Bare React Native
OTA Updates	Built-in (EAS Update)	Manual (CodePush)
Native Modules	Limited to Expo SDK	Full access
Build infra	EAS Build (cloud)	Local/CI
Frappe BLE/USB sync	Not supported	Supported

Decision: Expo managed workflow, with a bare eject plan for Phase 3.

Reasoning: The first two months focus on authentication, content delivery, and AI tutor none of these require native modules beyond what Expo provides. Ejecting is a one-

command operation (npx expo eject) when offline Bluetooth sync with local DIKSHA servers is needed. Starting bare adds overhead without benefit.

1.2 Application Layer Architecture



1.3 State Management: Why Zustand over Redux

Redux is the standard recommendation but introduces unnecessary boilerplate for a team where the contributor is the sole implementor. Zustand gives:

- Minimal setup (no actions/reducers/selectors split)
- Built-in middleware for AsyncStorage persistence

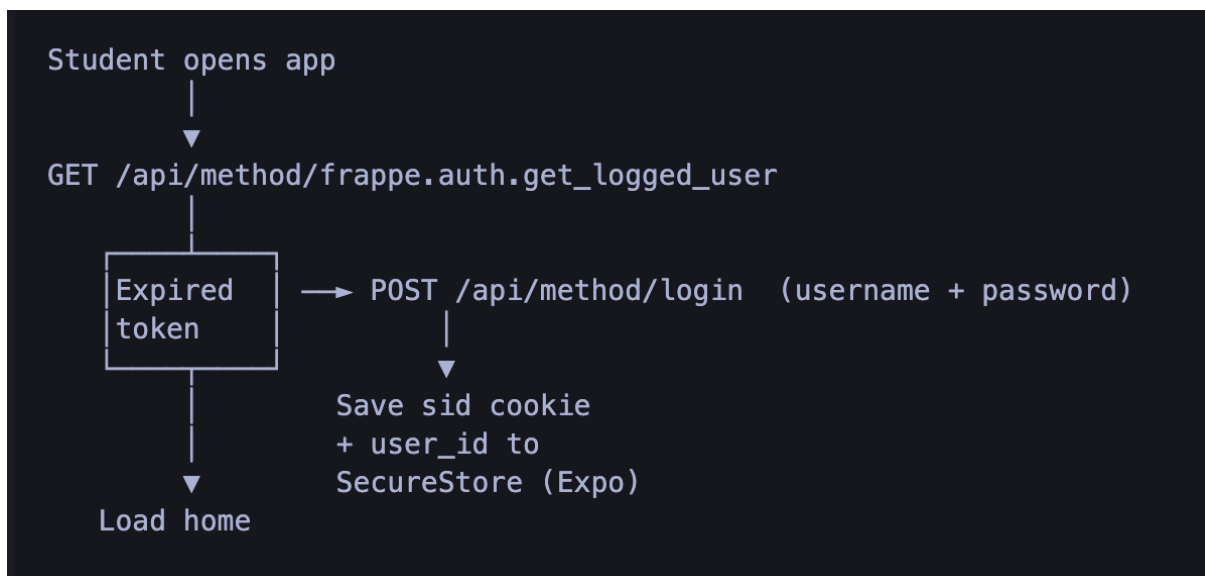
- Devtools parity with Redux DevTools via zustand/middleware
- Simpler async patterns (no redux-thunk/saga)

The tutorStore is the only slice needing complex middleware (WebSocket message queue + optimistic UI), and Zustand handles this cleanly with a custom WebSocket middleware.

Part 2: Authentication & Student Onboarding

2.1 Frappe Auth Flow

Frappe LMS exposes `/api/method/frappe.auth.get_logged_user` and `/api/method/login`. The native app will use:



Token stored in expo-secure-store (encrypted keychain on Android/iOS), never in AsyncStorage. On app cold start, the sid is re-validated before any API call; if expired, a silent refresh is attempted using the stored credentials before showing the login screen.

2.2 Onboarding Flow (Low-Literacy Design Principles)

- **Step 1:** Phone number + OTP (avoids password literacy barrier)
- **Step 2:** School selection via searchable district → block → school hierarchy
- **Step 3:** Class/Grade selection (large visual cards, no text required)
- **Step 4:** Language preference (auto-detected from device locale, user can change)
- **Step 5:** Avatar selection (identity formation, improves retention — backed by TAP research)

All onboarding steps are persisted to the offline DB immediately; if the student loses connectivity mid-onboarding, they resume exactly where they left off.

Part 3: Content Delivery

3.1 Frappe LMS Content API Mapping

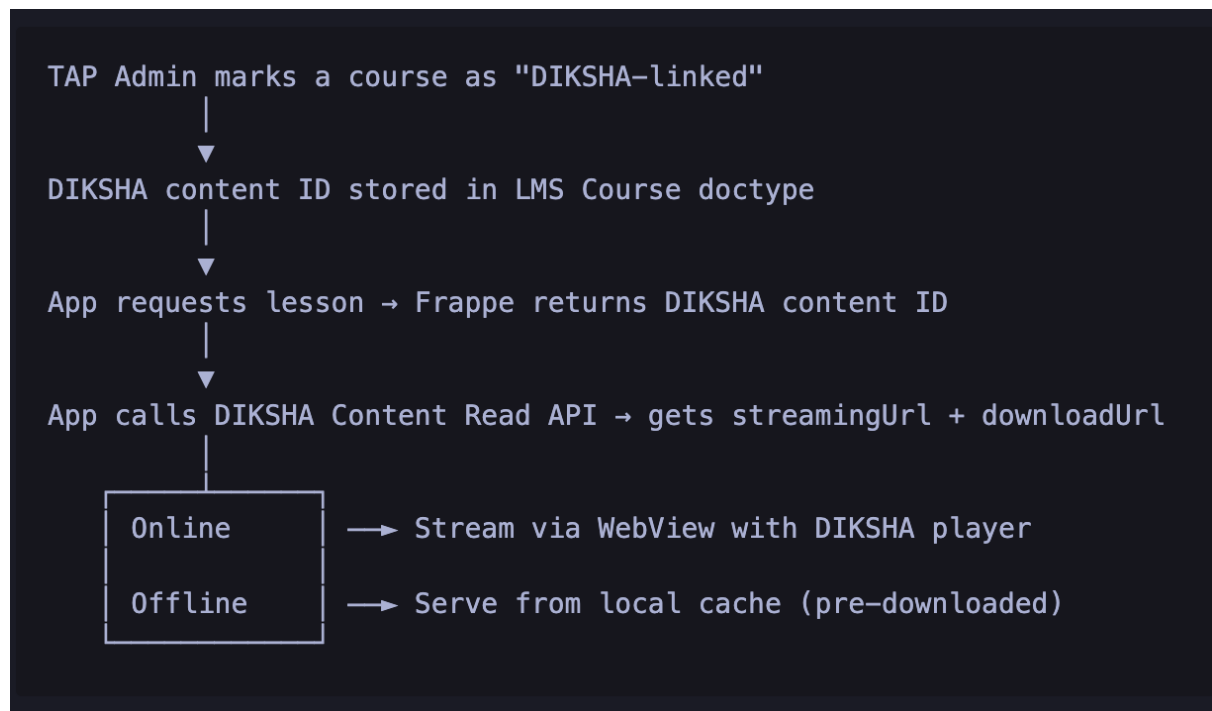
The existing TAP LMS (Frappe-based) exposes these endpoints that the app will consume:

Endpoint	Purpose	Caching strategy
GET /api/resource/LMS Course	Course catalog	24h cache, background refresh
GET /api/resource/LMS Lesson	Lesson content (HTML/video)	Persistent offline (WatermelonDB)
GET /api/resource/LMS Quiz	Assessment questions	Persistent offline
POST /api/resource/LMS Course Enrollment	Enroll student	Online-required, queued if offline
POST /api/resource/LMS Quiz Submission	Submit answers	Queued, synced on reconnect

3.2 DIKSHA Integration

DIKSHA is India's national school education platform operated by NCERT/MHRD. It exposes a public Content API (api.diksha.gov.in/api/content/v1/search) returning ECML/H5P content objects.

Integration architecture:



The DIKSHA player is an Angular web app embedded via WebView. To avoid white-screen on load, I will pre-warm the WebView during app boot and inject content directly via `window.postMessage` rather than full navigation.

3.3 Offline Sync with WatermelonDB

WatermelonDB is a high-performance reactive database for React Native built on SQLite. It outperforms AsyncStorage by 10–100x on large datasets (tested with 10k records)

Below is DB Schema attached.

```
// db/schema.ts
const courseSchema = tableSchema({
  name: 'courses',
  columns: [
    { name: 'frappe_id', type: 'string', isIndexed: true },
    { name: 'title', type: 'string' },
    { name: 'description', type: 'string' },
    { name: 'thumbnail_url', type: 'string' },
    { name: 'is_downloaded', type: 'boolean' },
    { name: 'last_synced_at', type: 'number' },
  ],
});

const progressSchema = tableSchema({
  name: 'progress',
  columns: [
    { name: 'lesson_id', type: 'string', isIndexed: true },
    { name: 'student_id', type: 'string', isIndexed: true },
    { name: 'completion_pct', type: 'number' },
    { name: 'is_synced', type: 'boolean' }, // false = pending
  ],
  server sync: {
    { name: 'updated_at', type: 'number' },
  },
});
```

Sync strategy: Pull-based delta sync on app foreground. Any progress row where `is_synced = false` is batched and POSTed to Frappe on reconnect. Conflict resolution rule: server wins for content; client wins for progress (student's local progress is never overwritten by stale server data).

Part 4: Real-Time AI Tutor via WebSocket

4.1 Current TAP AI Architecture (from reading the Ai repo)

The existing Ai repository is a Frappe app that:

1. Receives a student query via HTTP POST
2. Routes it through a RAG pipeline (rag_service)
3. Returns a completion from an LLM (currently Gemini)

The HTTP round-trip adds 800–1200ms of overhead just in connection setup. For a conversational tutor, this makes the experience feel broken.

4.2 WebSocket Upgrade Plan

Frappe supports Socket.IO via `frappe.realtime`. I will:

1. Open a Socket.IO connection on app mount (authenticated via the same sid cookie)
2. Emit `tutor_query` events from the app
3. Listen for `tutor_chunk` events (streaming response — one token at a time)
4. Assemble chunks into the chat message UI in real-time

```
// api/websocket.ts
import { io, Socket } from 'socket.io-client';

class TutorSocket {
  private socket: Socket | null = null;

  connect(siteUrl: string, sid: string) {
    this.socket = io(siteUrl, {
      extraHeaders: { Cookie: `sid=${sid}` },
      transports: ['websocket'],
    });

    this.socket.on('tutor_chunk', (data: { token: string; done: boolean }) => {
      tutorStore.getState().appendChunk(data.token, data.done);
    });
  }

  sendQuery(message: string, courseContext: string) {
    this.socket?.emit('tutor_query', { message, course_id: courseContext });
  }

  disconnect() {
    this.socket?.disconnect();
  }
}

export const tutorSocket = new TutorSocket();
```

The UI uses a streaming text component that renders tokens as they arrive, giving a ChatGPT-like experience on sub-100ms chunk intervals.

4.3 Offline Tutor Fallback

When the device is offline:

- Last 20 tutor conversation turns are persisted to WatermelonDB
- A local "hint engine" surfaces pre-generated hints for the current lesson (stored as part of lesson metadata)
- A clear "You're offline — showing cached hints" indicator replaces the live tutor

Part 5: Whitelabeling System

Multiple NGOs and state departments will deploy TAP Buddy under their own brand. The current web app has no whitelabeling. I will implement:

5.1 Zero-Code Whitelabeling Architecture

```

whitelabel/
├── default/
│   └── config.json      ← brand colors, logo, app name, feature
├── flags
│   ├── logo.png
│   └── splash.png
├── pratham/
│   ├── config.json
│   ├── logo.png
│   └── splash.png
├── diksha-maharashtra/
│   └── config.json
└── ...

```

```

// whitelabel/pratham/config.json
{
  "appName": "Pratham Learn",
  "primaryColor": "#E84C1E",
  "secondaryColor": "#FFF3EC",
  "logoUri": "./logo.png",
  "splashUri": "./splash.png",
  "features": {
    "voiceTutor": false,
    "dikshaContent": true,
    "parentNotifications": true
  },
  "lmsBaseUrl": "https://lms.pratham.org",
  "supportPhone": "1800-XXX-XXXX"
}

```

At EAS Build time, the correct partner slug is injected as an environment variable (EXPO_PUBLIC_PARTNER_SLUG=pratham). The app reads this at runtime and loads the correct whitelabel/[slug]/config.json. No code changes required per partner — a new deployment is one eas build --profile pratham command.

5.2 Theme Propagation

A ThemeProvider wraps the entire app and injects brand colors into a React Context. All UI components consume useTheme() instead of hardcoded hex values. This means changing primaryColor in one JSON file re-skins the entire app.

Part 6: Government Compliance & Security

For a GovTech deployment (especially DIKSHA integration), several compliance requirements apply:

Requirement	Implementation
No PII in logs	Strip student_id, phone, name from all console/Sentry logs in production builds
Data residency	All API calls route through TAP's India-region servers; no US-origin API calls with student data
WCAG 2.1 AA	Minimum 4.5:1 color contrast, accessibilityLabel on all interactive elements, screen reader testing with TalkBack
Offline data encryption	WatermelonDB + SQLCipher (AES-256) for the local database file
Secure credentials	expo-secure-store (Android Keystore / iOS Keychain) for auth tokens — never AsyncStorage
App signing	Play Store release signing managed via EAS Credentials

Part 7: Testing Strategy

Testing is integrated into every phase, not bolted on at the end
 Imp : We can tweak this at our will and as per need.

Unit Tests (Jest + React Native Testing Library)

- authStore: login/logout/token-refresh flows
- syncQueue: offline mutation queue correctness
- TutorSocket: mock Socket.IO, verify chunk assembly
- whitelabelConfig: config loading, feature flag gating

Integration Tests (Detox)

- Student onboarding end-to-end on Android emulator
- Login → Enroll → Start Lesson → Submit Quiz happy path
- Offline mode: disable network, verify content available, re-enable, verify sync

Manual QA Checkpoints (per PR)

- Screenshots at 360px/412px/768px (budget Android → tablet)
- TalkBack screen reader walkthrough on onboarding + lesson flow
- Low-bandwidth simulation (Chrome DevTools Network tab equivalent in emulator: 2G preset)

5.TimeLine:-

Break down the entire project into chunks and tell us what will you work on each week.

Weeks	Milestone	Tasks to be Completed
Pre-DMP (Now – June 9)	Codebase Deep Dive	Read all TAP repos in detail. Set up local Frappe LMS dev instance. Map all REST endpoints. Draft Figma wireframes for onboarding + lesson flows. PR: fix any existing issues found in TAP repos.
Week - 1 (10th June – 16th June)	Project Scaffold	- Expo project initialized with TypeScript strict mode. - Expo Router file-system routing configured. - Zustand stores scaffolded (auth, content, tutor). - ESLint + Prettier + Husky pre-commit. CI: GitHub Actions running jest on every PR.
Week - 2 (17th June – 23th June)	Authentication	Frappe REST client (/api/method/login, get_logged_user). SecureStore token persistence. Phone OTP login screen. All auth flows unit-tested.
Week - 3 (24th June – 30th June)	Onboarding	5-step onboarding flow (phone → school → grade → language → avatar). WatermelonDB schema defined and migrated. - Onboarding state persisted offline (resume on reconnect).
Week-4 (July 1st – 7th)	Content Catalog	- Course list + detail screens from Frappe LMS API. 24h cache layer. Enrollment API integration. Background image pre-fetch for thumbnails.
Week-5 (July 8th – 14th)	Offline Sync Content	WatermelonDB sync engine (pull delta from Frappe). "Download for offline" button on course detail. Progress tracking (local first, queued sync). Conflict resolution logic.

Week-6 (July 15th – 21)	AI Tutor v1 + DIKSHA	- WebSocket connection to Frappe Socket.IO. Streaming chat UI (token-by-token rendering). Offline tutor fallback (cached hints). DIKSHA Content Search API integration (search + display DIKSHA content in WebView). Mid-point demo: working app with auth, offline lessons, and streaming AI tutor in Hindi.
Week-7 (July 22 – 28)	DIKSHA deep integration	Pre-download DIKSHA content for offline play. DIKSHA WebView warm-up optimization. Content progress reported back to both TAP LMS and DIKSHA (where API allows).
Week-8 (July 29th – Aug 4)	Whitelabeling system	whitelabel/ directory structure. ThemeProvider context. EAS Build profiles per partner. config.json schema + validation. Three partner configs: default, pratham, diksha-mh.
Week-9 (Aug 5 – 11)	Progress & Gamification	Progress screen (lesson completion bars, quiz scores, streaks). expo-notifications for daily learning reminders. Parent view (simplified read-only progress dashboard).
Week-10 (Aug 12th – 18)	Accessibility + Compliance	TalkBack testing + fixes. Color contrast audit (all 3 whitelabel themes). SQLCipher encryption for WatermelonDB. PII scrubbing from logs. WCAG 2.1 AA documentation per screen.
Week-11 (Aug 19- 25)	Detox E2E + QA	Full Detox suite: onboarding, lesson, quiz, offline, sync flows. Performance profiling (Flipper): startup time < 2s on Redmi 9A (target device). Memory leak audit on tutor WebSocket.
Week-12 (Aug 26th – Sept 2nd)	Deployment	- EAS Build production profile. Play Store internal testing track submission. Partner deployment guide (how to create a new whitelabel config + build). Frappe endpoint documentation. - README with full local setup instructions (Frappe + React Native). Operational runbooks:

		crash triage procedure (Sentry → reproduce → hotfix), push notification delivery monitoring, offline sync failure alerting, and Play Store rollout strategy (staged rollout to 10% → 50% → 100%).
Final Evaluation (2 sept – 10 Sept)	Polish + Handover	Address all mentor review feedback. Final performance pass. Blog post / demo video. Final pytest/jest run — all tests green. Handover document: architecture decisions, known limitations, future roadmap items.

6. Some Important Questions:-

How many hours will you spend each week on your project?

During my summer vacation, I anticipate being able to dedicate approximately 20-30 hours per week to DMP-related activities. I am fully prepared to devote most of my time to DMP.

How will you report progress between evaluations?

I will maintain an active presence on GitHub by regularly submitting pull requests. Additionally, **I will actively engage on social media platforms such as LinkedIn, Instagram, and Twitter, where I will share updates on my progress, interact with the community, and promote participation in the project.**

7. Appendix: Key Files I've Read Thouroughly : -

1. theapprenticeproject/Ai — Frappe conversational routing app, all Python files
2. theapprenticeproject/rag_service — RAG pipeline, query handler, Frappe integration
3. theapprenticeproject/TAP-Lms-TAP-Trans — LMS doctype structure, enrollment logic
4. theapprenticeproject/evalix_web — Student-facing evaluation UI, component patterns
5. theapprenticeproject/TAP_AI4All — AI-for-all initiative context
6. Frappe LMS REST API docs — /api/resource/* patterns, session management

7. WatermelonDB sync docs — pull-only vs push-pull sync, SQLite schema migrations
8. EAS Build + Update docs — environment variable injection, partner build profiles
9. DIKSHA Content API docs — Content Search v1, Content Read v1, ECML spec
10. Expo Router v3 docs — file-system routing, layout groups, typed routes

Code for Good Tech

Community Driven Social Impact

Thankyou !!

Proposal by Anurag Singh (Oval17) | DMP | TAP